Part 1: Theoretical Questions

Question 1.1

1. {f:(T1->T2), g:(T1->T2), a:T1} |- (f (g a)) : T2

FALSE.

- a : T1 and g : (T1->T2), so (g a) : T2.
- f : (T1->T2) expects an argument of type T1, but it is applied to (g a) : T2.
- The application rule requires the argument type to equal f's domain, i.e., it requires T1 = T2. Since T1 and T2 are distinct (arbitrary) type variables, the term (f (g a)) is not typable. The judgment only holds if T1 = T2.

2. {f:(T1*T2->T3)} |- (lambda (x) (f x y)) : (T2->T3)

FALSE. Two problems:

- y is a free variable that does not appear in the type environment, so the body (f x y) cannot be typed at all under {f:(T1*T2->T3)}.
- Even ignoring that: in (f x y), x is the first argument of f, so x : T1 and the body has type T3. Hence the lambda has type (T1 -> T3), not (T2 -> T3).

3. {f:(T1*T2->T3), y:T2} |- (lambda (x) (f x y)) : (T1->T3)

TRUE.

- y : T2 is now in the environment.
- In (f x y): x is f's first argument so x : T1, y : T2 matches f's second argument, and the result is T3.
- The lambda binds x : T1 and its body has type T3, so its overall type is (T1 -> T3). Matches the claim.

Question 1.2

1. (lambda (f1 g1 x1 y1) (f1 (g1 y1 x1) (g1 x1 y1)))

i. Pool

Sub-expression	Type var
(lambda (f1 g1 x1 y1) ...) (whole)	T0
f1	Tf
g1	Tg
x1	Tx
y1	Ty
(f1 (g1 y1 x1) (g1 x1 y1)) (body)	Tb
(g1 y1 x1)	T1
(g1 x1 y1)	T2

ii. Equations

T0 = (Tf * Tg * Tx * Ty -> Tb) ; lambda Tf = (T1 * T2 -> Tb) ; (f1 (g1 y1 x1) (g1 x1 y1)) Tg = (Ty * Tx -> T1) ; (g1 y1 x1) Tg = (Tx * Ty -> T2) ; (g1 x1 y1)

iii. The inference succeeds.

Unifying the two equations for Tg: (Ty * Tx -> T1) = (Tx * Ty -> T2) => Ty = Tx and T1 = T2.

Final substitution: Ty := Tx T2 := T1 Tg := (Tx * Tx -> T1) Tf := (T1 * T1 -> Tb) T0 := ((T1 * T1 -> Tb) * (Tx * Tx -> T1) * Tx * Tx -> Tb)

Inferred type of the expression: ((T1 * T1 -> Tb) * (Tx * Tx -> T1) * Tx * Tx -> Tb)

---

2. ((lambda (f1) (lambda (x1) (f1 x1 x1))) (lambda (y1) y1))

i. Pool

Sub-expression	Type var
((lambda (f1) ...) (lambda (y1) y1)) (whole app)	Tapp
(lambda (f1) (lambda (x1) (f1 x1 x1)))	TL1
f1	Tf
(lambda (x1) (f1 x1 x1))	TL2
x1	Tx
(f1 x1 x1)	Tbd
(lambda (y1) y1)	Tid
y1	Ty

ii. Equations

TL1 = (Tid -> Tapp) ; outer application TL1 = (Tf -> TL2) ; (lambda (f1) ...) TL2 = (Tx -> Tbd) ; (lambda (x1) ...) Tf = (Tx * Tx -> Tbd) ; (f1 x1 x1) Tid = (Ty -> Ty) ; (lambda (y1) y1)

iii. The inference fails.

From the two TL1 equations: Tf = Tid. But Tf = (Tx * Tx -> Tbd) (a two-argument procedure type) and Tid = (Ty -> Ty) (a one-argument procedure type).

Unification reaches the conflicting equation: (Tx * Tx -> Tbd) = (Ty -> Ty)

This is an arity mismatch — a 2-parameter procedure type cannot unify with a 1-parameter procedure type.

---

3. ((lambda (f1) (lambda (x1) ((f1 x1) x1))) (lambda (y1) y1))

i. Pool

Sub-expression	Type var
((lambda (f1) ...) (lambda (y1) y1)) (whole app)	Tapp
(lambda (f1) (lambda (x1) ((f1 x1) x1)))	TL1
f1	Tf
(lambda (x1) ((f1 x1) x1))	TL2
x1	Tx
((f1 x1) x1) (outer app)	Tbd
(f1 x1) (inner app)	Tin
(lambda (y1) y1)	Tid
y1	Ty

ii. Equations

TL1 = (Tid -> Tapp) ; outer application TL1 = (Tf -> TL2) ; (lambda (f1) ...) TL2 = (Tx -> Tbd) ; (lambda (x1) ...) Tin = (Tx -> Tbd) ; ((f1 x1) x1) Tf = (Tx -> Tin) ; (f1 x1) Tid = (Ty -> Ty) ; (lambda (y1) y1)

iii. The inference fails.

From the two TL1 equations: Tf = Tid. With Tf = (Tx -> Tin) and Tid = (Ty -> Ty), unifying gives Tx = Ty and Tin = Ty, hence Tin = Tx.

Substituting into Tin = (Tx -> Tbd) yields the conflicting equation: Tx = (Tx -> Tbd)

This fails the occurs check — Tx would have to be a type that contains itself (an infinite type), which is untypable.

Question 1.3: Sum Types

Expression: (lambda (f) (f (right (lambda (s) (f (left s))))))

i. Pool

Sub-expression	Type var
(lambda (f) (f (right ...))) (whole)	T0
f	Tf
(f (right (lambda (s) (f (left s))))) (body)	T1
(right (lambda (s) (f (left s))))	T2
(lambda (s) (f (left s)))	T3
s	Ts
(f (left s))	T4
(left s)	T5

Fresh type variables introduced by the sum-type rules: Tr (from right), Tl (from left).

ii. Equations

T0 = (Tf -> T1) ; outer lambda Tf = (T2 -> T1) ; (f (right ...)) T2 = (Tr + T3) ; right rule (Tr fresh) T3 = (Ts -> T4) ; (lambda (s) ...) Tf = (T5 -> T4) ; (f (left s)) T5 = (Ts + Tl) ; left rule (Tl fresh)

iii. The inference succeeds.

Unifying the two Tf equations: (T2 -> T1) = (T5 -> T4) => T2 = T5 and T1 = T4.

Then T2 = T5 with T2 = (Tr + T3) and T5 = (Ts + Tl) gives Tr = Ts and T3 = Tl. From T3 = (Ts -> T4) and T1 = T4, we get Tl = T3 = (Ts -> T1).

Final substitution: T4 := T1 Tr := Ts Tl := (Ts -> T1) T3 := (Ts -> T1) T5 := (Ts + (Ts -> T1)) T2 := (Ts + (Ts -> T1)) Tf := ((Ts + (Ts -> T1)) -> T1) T0 := (((Ts + (Ts -> T1)) -> T1) -> T1)

Inferred type of the expression: (((Ts + (Ts -> T1)) -> T1) -> T1)